

Propuesta de diseño para un asistente inteligente de borde para errores DRC en celdas estándar VLSI

Alexandra Alfaro Elizondo*, Jose Pablo Esquetini Fallas*, Kendall Madrigal Campos*

*Escuela de Ingeniería Electrónica, Instituto Tecnológico de Costa Rica (ITCR), 30101 Cartago, Costa Rica,
{alexvaneska, jpesquetinif, kendallmacampos2941}@estudiantec.cr

Resumen

Se propone el diseño de un asistente inteligente de borde para la interpretación y corrección de errores de verificación de reglas de diseño (DRC) en celdas estándar VLSI, orientado al curso de Diseño mediante VLSI del Instituto Tecnológico de Costa Rica. El sistema implementa una arquitectura de Generación Aumentada por Recuperación (RAG) sobre la plataforma NVIDIA Jetson Nano, utilizando el modelo de lenguaje Phi-3 mini con cuantización Q4 y el motor de inferencia llama.cpp para operar de forma completamente local y sin conectividad a internet. La base de conocimiento se construye a partir de la documentación técnica del PDK XFAB XH018 de 180 nm, indexada mediante embeddings vectoriales en ChromaDB. El sistema operativo es sintetizado con Yocto Project y la aplicación se despliega en un contenedor Docker, garantizando reproducibilidad y portabilidad. Los requerimientos definidos establecen un tiempo de respuesta máximo de 60 segundos y una precisión mínima del 80 % sobre casos de prueba documentados, dentro de un presupuesto de memoria de 4 GB RAM.

Palabras Clave

DRC, Edge AI, Jetson Nano, LLM, Retrieval Augmented Generation, sistemas embebidos, VLSI, Yocto Project

I. INTRODUCCIÓN

En los últimos años, los sistemas de asistencia virtual y chatbots han pasado de ser herramientas simples a asistentes sofisticados capaces de entablar conversaciones similares a las humanas [1]. Este avance ha sido impulsado por los Modelos de Lenguaje de Gran Tamaño (LLMs), los cuales, sin embargo, enfrentan retos críticos como la desactualización de la información y la falta de conocimientos específicos de un dominio técnico [1]. Para solventar estas limitaciones, la arquitectura de Generación Aumentada por Recuperación (RAG) ha emergido como un paradigma esencial, permitiendo que el modelo recupere información relevante de fuentes externas para generar respuestas precisas y contextualizadas [1], [2].

Paralelamente, existe una tendencia creciente hacia la Inteligencia Artificial de Borde (Edge AI), que busca desplazar la inferencia de modelos generativos desde nubes centralizadas hacia dispositivos locales [3]. Este cambio es fundamental para abordar problemas de privacidad de datos, cuellos de botella por latencia y altos costos operativos [3]. El uso de plataformas como la NVIDIA Jetson Nano permite ejecutar estos sistemas inteligentes de forma autónoma, sin dependencia de internet, lo que resulta ideal para aplicaciones críticas o de soporte técnico en tiempo real [2], [4], [5].

II. JUSTIFICACIÓN

La implementación de asistentes inteligentes locales se justifica por la necesidad de ofrecer soluciones rápidas y seguras en entornos de hardware restringido. Los métodos tradicionales de configuración y soporte técnico suelen depender de esfuerzos manuales que consumen mucho tiempo y son propensos a errores [2]. La integración de contenedores como Docker en estos flujos de trabajo garantiza un entorno consistente y aislado, lo cual es vital para dispositivos con recursos limitados como la Jetson Nano [2].

Además, la adopción de tecnologías de borde no solo mejora la eficiencia operativa, sino que también promueve la sostenibilidad. El uso de hardware especializado y eficiente energéticamente permite un equilibrio entre la velocidad de generación de tokens y la precisión semántica, sin los costos ambientales asociados al procesamiento masivo en la nube [6]. Asimismo, en dominios donde la seguridad es primordial, como la protección de infraestructuras IoT, la capacidad de realizar análisis de datos locales reduce significativamente la exposición a ciberataques y garantiza la integridad de la información técnica [4]. Por tanto, el desarrollo de un asistente basado en RAG y desplegado en el borde representa una solución robusta para democratizar el acceso a conocimiento especializado de forma privada y eficiente [1], [6], [7].

III. DESCRIPCIÓN DEL PROBLEMA

El diseño de circuitos integrados mediante herramientas de automatización electrónica (EDA, por sus siglas en inglés) constituye una disciplina técnica de alta complejidad, cuyo dominio requiere no solo comprensión teórica de los principios del diseño VLSI, sino también experiencia práctica en la interpretación y corrección de errores generados durante el proceso de verificación. En el contexto del curso de Diseño mediante VLSI del Instituto Tecnológico de Costa Rica, los estudiantes

utilizan Cadence Virtuoso junto con el PDK XFAB XH018 para diseñar y verificar celdas estándar digitales. Una parte fundamental de este proceso es la verificación de reglas de diseño, conocida como DRC (Design Rule Check), la cual garantiza que el layout físico de un circuito cumpla con las restricciones geométricas impuestas por el proceso de fabricación. [8]

Durante las sesiones de laboratorio, es común que los estudiantes generen múltiples errores DRC al diseñar sus celdas. Estos errores son reportados por la herramienta Calibre en forma de logs técnicos que incluyen identificadores de reglas, coordenadas de violación y valores numéricos de las restricciones incumplidas. Sin embargo, la interpretación de estos mensajes no es trivial: cada error hace referencia a una regla específica del PDK, cuya comprensión requiere consultar la documentación técnica del proceso de fabricación, identificar la capa afectada, entender el tipo de violación y determinar la corrección adecuada dentro del entorno de Virtuoso. Este proceso de interpretación y corrección representa una barrera significativa para estudiantes que se encuentran en etapas tempranas de aprendizaje del flujo de diseño VLSI.

La situación se agrava por la naturaleza del entorno educativo. En un laboratorio con múltiples estudiantes trabajando simultáneamente, la disponibilidad del profesor para atender consultas individuales es limitada. Cada sesión de laboratorio impone restricciones de tiempo que dificultan que el estudiante pueda detenerse a consultar extensamente la documentación del PDK, la cual en el caso del XFAB XH018 no es de acceso público y requiere navegar documentos técnicos densos y especializados. Esta combinación de factores genera ciclos de frustración y retraso que afectan directamente el ritmo de aprendizaje y la calidad de los diseños producidos por los estudiantes. Además de que cuando trabajan desde casa el acceso a una guía por parte del profesor pasa a ser más complicada debido a la comunicación mediante correos con el profesor.

Desde una perspectiva más amplia, el problema refleja una brecha existente en la integración de herramientas de inteligencia artificial en entornos educativos de diseño de circuitos integrados. Si bien los modelos de lenguaje de gran escala han demostrado capacidad para asistir en tareas técnicas complejas, su adopción en contextos de diseño VLSI ha sido limitada, particularmente en entornos con restricciones de conectividad, privacidad de datos o recursos computacionales. Las soluciones basadas en servicios de IA en la nube presentan dependencias de conectividad a internet, latencias variables y riesgos de exposición de diseños propietarios, lo que las hace inadecuadas para entornos de laboratorio donde la privacidad del trabajo estudiantil y la disponibilidad del servicio son requisitos críticos. [9], [10]

En este contexto surge la necesidad de una solución de inteligencia artificial de borde, capaz de operar de forma completamente local sobre hardware embebido de bajo costo, que pueda asistir al estudiante en la interpretación y corrección de errores DRC de manera inmediata, contextualizada al PDK y al entorno de herramientas específicos del curso, sin depender de servicios externos ni comprometer la privacidad de los diseños. Esta solución debe ser suficientemente liviana para ejecutarse en plataformas como la Jetson Nano, suficientemente precisa para responder dentro del dominio técnico del PDK XFAB XH018, y suficientemente accesible para ser utilizada por estudiantes durante sus sesiones de laboratorio sin requerir conocimientos adicionales de inteligencia artificial. [3]

La ausencia de esta herramienta en el entorno actual del curso representa un problema tanto pedagógico como tecnológico: pedagógico porque limita la autonomía del estudiante y ralentiza su curva de aprendizaje en una etapa crítica de su formación como ingeniero en electrónica, y tecnológico porque evidencia la falta de integración de sistemas inteligentes de borde en flujos de trabajo de diseño VLSI en entornos académicos, un área con amplio potencial de desarrollo e impacto en la industria de semiconductores.

IV. REQUERIMIENTOS DEL SISTEMA

Los requerimientos presentados a continuación fueron derivados del análisis del problema descrito en la sección anterior, del concepto de operaciones propuesto y de los objetivos específicos establecidos en el instructivo del proyecto. Se clasifican en requerimientos funcionales (FR), requerimientos no funcionales (NFR) y restricciones (CON), organizados por módulo del sistema.

IV-A. *Requerimientos funcionales*

IV-A1. *Módulo 1 — Gestión del conocimiento DRC:*

[FR-01] Carga de documentación DRC

El sistema deberá permitir al asistente del curso cargar documentación técnica del PDK XFAB XH018 en formato PDF o texto plano para su procesamiento e indexación.

Justificación: *La base de conocimiento del sistema depende de documentación técnica específica del PDK utilizado en*

el curso. Sin esta funcionalidad, el sistema no puede ser inicializado con información relevante para los estudiantes.

[FR-02] División de documentos en fragmentos

El sistema deberá dividir automáticamente los documentos cargados en fragmentos de entre 300 y 500 tokens, con un solapamiento de 50 tokens entre fragmentos adyacentes.

Justificación: La granularidad de los fragmentos determina la precisión de la recuperación de información. Un tamaño adecuado garantiza que cada fragmento contenga información coherente y completa sobre una regla DRC específica.

[FR-03] Indexación vectorial de fragmentos

El sistema deberá generar embeddings de cada fragmento mediante el modelo `all-MiniLM-L6-v2` y almacenarlos de forma persistente en una base de datos vectorial ChromaDB local.

Justificación: La indexación vectorial es el mecanismo que permite la recuperación semántica de información relevante ante una consulta del usuario, condición necesaria para el funcionamiento del pipeline RAG.

[FR-04] Actualización del corpus

El sistema deberá permitir al asistente del curso agregar, modificar o eliminar ejemplos y documentación de la base de conocimiento sin necesidad de reconstruir la base de datos completa.

Justificación: El corpus debe poder mejorarse de forma incremental a lo largo del semestre, incorporando correcciones señaladas por el profesor sin interrumpir la operación del sistema.

IV-A2. Módulo 2 — Pipeline RAG:

[FR-05] Procesamiento de consultas

El sistema deberá recibir una consulta en lenguaje natural o un error DRC en formato de log, generar su embedding y recuperar los fragmentos más relevantes de la base de conocimiento mediante búsqueda por similitud semántica.

Justificación: Este es el núcleo funcional del sistema RAG. Sin esta capacidad, el modelo de lenguaje no puede ser contextualizado con información específica del PDK XH018.

[FR-06] Construcción del prompt aumentado

El sistema deberá construir un prompt que combine los fragmentos recuperados con la consulta del usuario antes de enviarlo al modelo de lenguaje.

Justificación: El prompt aumentado es el mecanismo que permite al LLM generar respuestas contextualizadas al dominio DRC sin necesidad de haber sido entrenado con esa información.

[FR-07] Notificación fuera de alcance

El sistema deberá notificar explícitamente al usuario cuando una consulta no encuentre contexto relevante en la base de conocimiento, indicando que el error está fuera del alcance definido del sistema.

Justificación: Una respuesta generada sin contexto relevante puede ser incorrecta o engañosa. La notificación explícita protege al estudiante de actuar sobre información incorrecta.

IV-A3. Módulo 3 — Modelo de lenguaje:

[FR-08] Generación de respuesta en lenguaje natural

El sistema deberá generar una respuesta en lenguaje natural que explique el error DRC consultado y proporcione los pasos específicos para su corrección en Cadence Virtuoso, basándose exclusivamente en el contexto recuperado.

Justificación: El valor principal del sistema para el estudiante es recibir una explicación comprensible del error y una guía de corrección accionable, no simplemente una referencia a la documentación técnica.

[FR-09] Ejecución local del modelo

El sistema deberá ejecutar el modelo de lenguaje Phi-3 mini mediante `llama.cpp` directamente sobre la Jetson Nano, sin realizar llamadas a servicios externos de inferencia.

Justificación: La operación local garantiza disponibilidad sin dependencia de conectividad a internet y preserva la privacidad de los diseños consultados por los estudiantes.

IV-A4. Módulo 4 — Interfaz de usuario:

[FR-10] Interfaz de consulta

El sistema deberá exponer una interfaz accesible mediante la terminal del sistema, que permita al estudiante y al profesor ingresar consultas y visualizar las respuestas generadas.

Justificación: Una interfaz en terminal no requiere instalación de software adicional en las estaciones de trabajo de los estudiantes, facilitando su adopción en el entorno de laboratorio.

[FR-11] Registro de consultas

El sistema debería registrar cada consulta recibida junto con la respuesta generada en un log local, para su revisión posterior por parte del asistente del curso y el profesor.

Justificación: El registro permite identificar patrones de errores frecuentes y evaluar la calidad del sistema a lo largo del semestre, apoyando el proceso de mejora continua del corpus.

IV-A5. Módulo 5 — Sistema operativo e infraestructura:

[FR-12] Síntesis de imagen Linux con Yocto Project

El sistema deberá ser desplegado mediante una imagen de Linux personalizada sintetizada con Yocto Project, que integre Ollama y todas las dependencias de software necesarias para la operación del pipeline RAG.

Justificación: El uso de Yocto Project garantiza una imagen mínima, reproducible y optimizada para el hardware de la Jetson Nano, reduciendo el consumo de recursos y facilitando el despliegue del sistema.

[FR-13] Gestión del modelo con Ollama

El sistema deberá utilizar Ollama como herramienta de gestión del ciclo de vida del modelo de lenguaje, incluyendo su descarga, configuración y exposición mediante una API local.

Justificación: Ollama simplifica la integración del modelo de lenguaje con el resto del pipeline al proporcionar una API local estandarizada, compatible con el entorno de hardware restringido de la Jetson Nano.

IV-B. Requerimientos no funcionales

[NFR-01] Tiempo de respuesta

El sistema deberá generar una respuesta completa en un tiempo máximo de 60 segundos desde la recepción de la consulta, bajo condiciones normales de operación en la Jetson Nano.

Justificación: Un tiempo de respuesta mayor reduciría la utilidad del sistema durante sesiones de laboratorio con tiempo limitado, afectando la experiencia del usuario.

[NFR-02] Operación sin conectividad

El sistema deberá operar de forma completamente local, sin requerir acceso a internet en ninguna etapa del pipeline de inferencia.

Justificación: El entorno de laboratorio no garantiza conectividad estable, y la dependencia de servicios externos comprometería la disponibilidad del sistema durante las sesiones.

[NFR-03] Consumo de memoria

El sistema deberá operar dentro de los 4 GB de memoria RAM disponibles en la Jetson Nano, considerando el modelo de lenguaje cuantizado, el modelo de embeddings y la base de datos vectorial de forma simultánea.

Justificación: La Jetson Nano comparte la memoria RAM entre CPU y GPU. Exceder este límite provocaría fallos de ejecución que harían inoperable el sistema.

[NFR-04] Precisión del dominio

El sistema deberá responder correctamente al menos el 80 % de las consultas sobre errores DRC dentro del alcance definido, validado mediante casos de prueba con soluciones conocidas.

Justificación: Una tasa de precisión inferior comprometería la confianza de los estudiantes en el sistema y podría inducirlos a aplicar correcciones incorrectas en sus diseños.

[NFR-05] Usabilidad

El sistema debería permitir que un estudiante sin conocimientos previos de inteligencia artificial pueda realizar su primera consulta exitosa en menos de 5 minutos desde el acceso a la interfaz.

***Justificación:** El sistema está dirigido a estudiantes de ingeniería electrónica cuya formación principal no es en inteligencia artificial. Una interfaz compleja reduciría su adopción en el laboratorio.*

[NFR-06] Privacidad de datos

El sistema deberá procesar todas las consultas de forma local sin transmitir información de diseño a servicios externos, garantizando la privacidad de los trabajos estudiantiles.

***Justificación:** Los diseños desarrollados en el curso pueden contener propiedad intelectual académica. Su transmisión a servicios externos representaría un riesgo de confidencialidad inaceptable.*

IV-C. Restricciones

[CON-01] Plataforma de hardware

El sistema deberá ejecutarse exclusivamente sobre la plataforma Jetson Nano con 4 GB de memoria RAM, sin posibilidad de escalar a hardware adicional durante la demostración del proyecto.

[CON-02] PDK objetivo

El sistema deberá estar delimitado al PDK XFAB XH018 de 180 nm. No se contempla soporte para otros procesos de fabricación dentro del alcance de este proyecto.

[CON-03] Celdas cubiertas

El sistema deberá cubrir únicamente los errores DRC asociados al diseño de celdas estándar NOT, AND y NOR. El soporte para otras celdas queda fuera del alcance definido.

[CON-04] Capas y tipos de error

El sistema deberá dar soporte únicamente a errores de tipo Spacing, Width, Enclosure, Extension y reglas de Via y Contact, sobre las capas Metal1 a Metal4, Poly, Ndiff, Pdiff, Contact y Via.

[CON-05] Herramienta de síntesis del sistema operativo

El sistema operativo de la Jetson Nano deberá ser sintetizado mediante Yocto Project, conforme a los requerimientos del instructivo del proyecto. No se permite el uso de imágenes Linux precompiladas como alternativa.

[CON-06] Modelo de lenguaje

El modelo de lenguaje utilizado deberá tener un máximo de 4 mil millones de parámetros en su versión cuantizada Q4, como restricción derivada de las limitaciones de memoria de la Jetson Nano.

V. VISTA OPERACIONAL

V-A. Concepto de Operaciones (ConOps)

El asistente inteligente de borde para errores DRC opera de forma continua y local sobre la plataforma Jetson Nano, disponible durante las sesiones de laboratorio del curso de Diseño mediante VLSI del Instituto Tecnológico de Costa Rica. El sistema no requiere conexión a internet, lo que garantiza disponibilidad inmediata dentro del entorno del laboratorio y privacidad total de los diseños trabajados por los estudiantes.

El ciclo de vida operacional del sistema comprende dos etapas diferenciadas. La primera es una etapa de preparación, ejecutada por el asistente del curso previo al inicio de las sesiones, en la cual se construye y actualiza la base de conocimiento DRC con documentación técnica del PDK XFAB XH018 y ejemplos de errores reales anotados. Esta etapa no requiere intervención de los estudiantes ni del profesor y se ejecuta una única vez por semestre, con actualizaciones puntuales según sea necesario.

La segunda etapa es la etapa de operación, que ocurre durante las sesiones de laboratorio. En esta fase, el estudiante trabaja en el diseño de celdas estándar básicas en Cadence Virtuoso y, al encontrar un error DRC en el log generado por Calibre, accede a la interfaz de terminal del asistente desde el mismo entorno de trabajo. El estudiante ingresa el error o formula una pregunta en lenguaje natural, y el sistema responde en cuestión de segundos con una explicación contextualizada del error y los pasos específicos para corregirlo dentro de Virtuoso, sin necesidad de consultar manualmente la documentación del PDK ni interrumpir al profesor.

El profesor, por su parte, interactúa con el sistema de forma periódica para validar la calidad de las respuestas generadas. Ante una respuesta deficiente, notifica al asistente del curso para que el corpus sea corregido, activando un ciclo de mejora continua que fortalece el sistema a lo largo del semestre.

El sistema está delimitado a los errores DRC más frecuentes en el diseño de celdas NOT, AND y NOR, sobre las capas Metal1 a Metal4, Poly, Ndiff, Pdiff, Contact y Via, cubriendo los tipos de error Spacing, Width, Enclosure, Extension y reglas de Via y Contact. Esta delimitación garantiza que el sistema responda con precisión dentro de su alcance definido y notifique explícitamente al usuario cuando una consulta esté fuera del mismo.

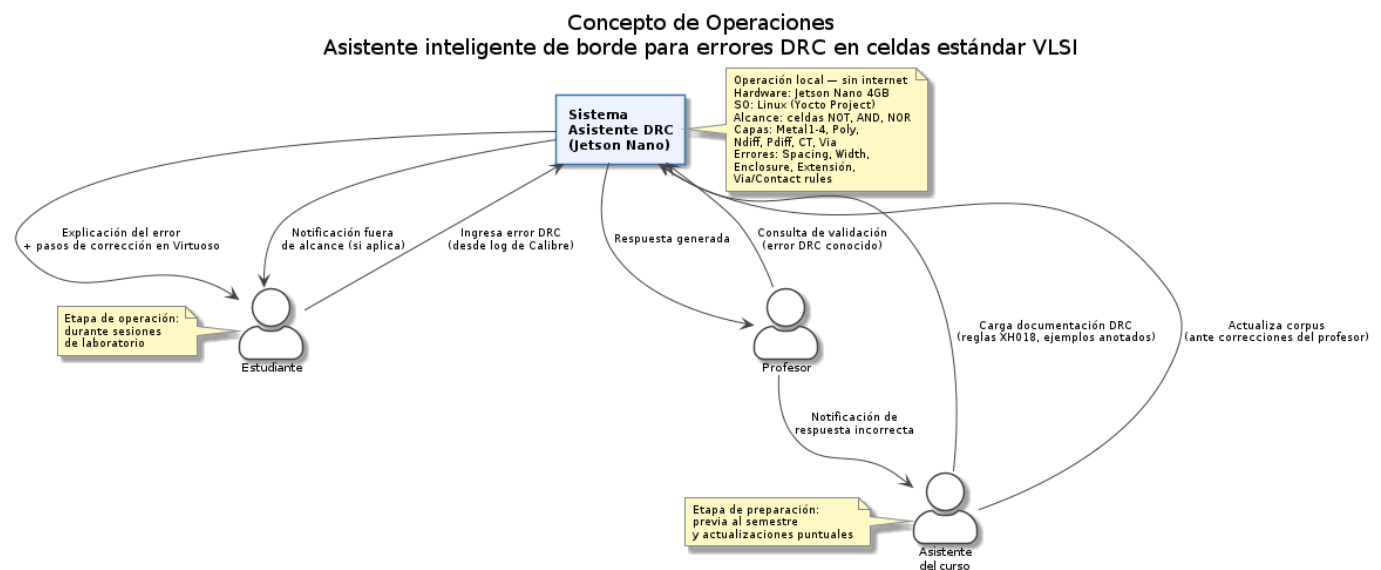


Figura 1. Concepto de operaciones del asistente inteligente de borde para errores DRC

V-B. Casos de uso

■ Caso de uso 1.

Identificador: CU01 - Consultar al asistente DRC

Actores: Estudiante, profesor

Propósito: Permitir al usuario obtener una explicación en lenguaje natural de un error DRC y los pasos para corregirlo en Virtuoso, a partir de la base de conocimiento del sistema.

Precondiciones: La base de conocimiento DRC está indexada y disponible, el sistema está corriendo en la Jetson Nano, el usuario tiene un error DRC generado durante la verificación en Cadence Virtuoso.

Postcondiciones: El sistema devuelve una explicación del error y pasos de corrección, la consulta queda registrada en el log del sistema.

Flujo principal: El usuario ingresa el error DRC o una consulta en lenguaje natural en la interfaz de terminal del sistema. El sistema genera el embedding de la consulta y recupera los fragmentos más relevantes de la base de conocimiento mediante búsqueda por similitud semántica. Si la similitud máxima es inferior al umbral definido, el sistema notifica que la consulta está fuera de alcance. En caso contrario, el sistema construye el prompt aumentado y lo envía al modelo Phi-3 mini, que genera la respuesta en lenguaje natural con la explicación del error y los pasos de corrección.

■ Caso de uso 2.

Identificador: CU02 - Gestionar base de conocimiento DRC

Actores: Asistente del curso

Propósito: Permitir al asistente del curso agregar, actualizar o eliminar ejemplos y documentación DRC en la base de conocimiento del sistema.

Precondiciones: El asistente tiene acceso al sistema con permisos de administración, los documentos o ejemplos a agregar están en formato compatible.

Postcondiciones: La base de conocimiento refleja los cambios realizados, los nuevos ejemplos están disponibles para consultas futuras.

Flujo principal: El asistente del curso accede al sistema en modo administración, el asistente agrega nuevos ejemplos DRC anotados o documentación, el sistema procesa e indexa el nuevo contenido, el sistema confirma que el contenido está disponible.

■ Caso de uso 3.

Identificador: CU03 - Supervisar respuestas del sistema

Actores: Profesor

Propósito: Permitir al profesor evaluar la calidad y precisión de las respuestas del sistema para validar su uso en el curso.

Precondiciones: CU01 se completó con una respuesta generada, el profesor tiene acceso al sistema.

Postcondiciones: El profesor ha evaluado la respuesta, opcionalmente el profesor notifica al asistente del curso sobre correcciones necesarias.

Flujo principal: El profesor realiza una consulta con un error DRC conocido, el sistema genera la respuesta vía CU01, el profesor compara la respuesta con la solución correcta conocida, el profesor determina si la respuesta es aceptable.

Diagrama de Casos de Uso

Asistente inteligente de borde para errores DRC en celdas estándar VLSI

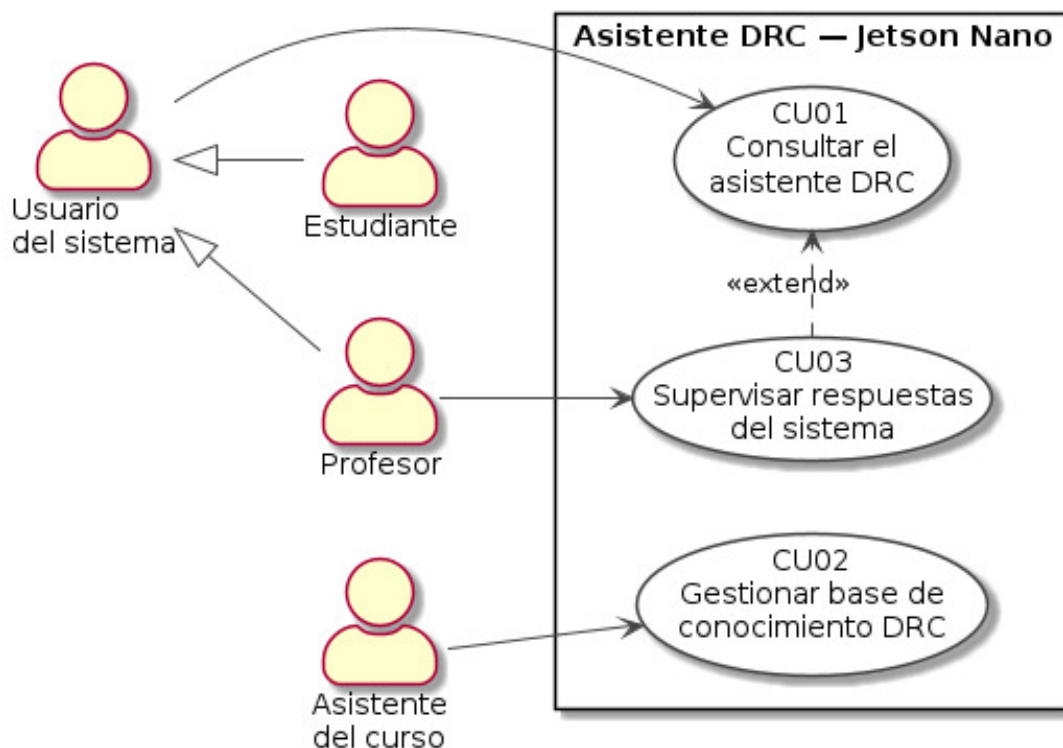


Figura 2. Diagrama de casos de uso

V-C. Flujo de trabajo por actor

V-C1. Flujo de trabajo del Asistente del curso: El asistente del curso cumple un rol de administrador del conocimiento del sistema. Su participación es fundamental antes de que el sistema entre en operación, ya que es el responsable de construir y mantener la base de conocimiento DRC sobre la cual opera el pipeline RAG. El proceso inicia con la recopilación de documentación técnica relevante, incluyendo las reglas de diseño del PDK XFAB XH018 y ejemplos de errores DRC anotados manualmente. Una vez recopilado el material, el asistente lo carga al sistema, donde es procesado, dividido en fragmentos y vectorizado para su almacenamiento en la base de datos local. Su rol no concluye con la indexación inicial: ante una notificación del profesor sobre una respuesta incorrecta del sistema, el asistente identifica la deficiencia en el corpus, corrige o agrega el ejemplo correspondiente y ejecuta la reindexación del contenido actualizado, garantizando la mejora continua del sistema a lo largo del curso.

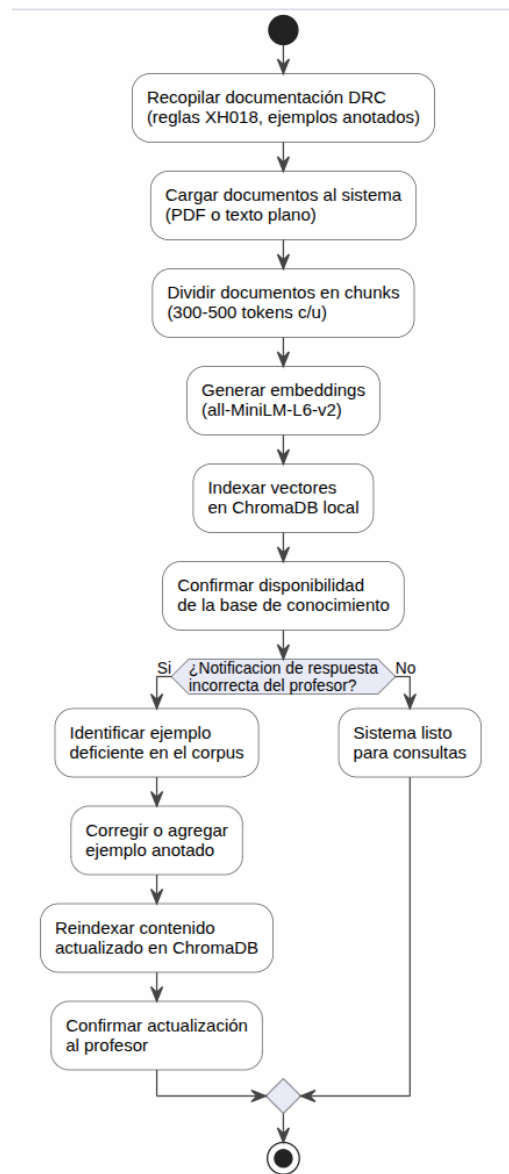


Figura 3. Flujo de trabajo del asistente del curso

V-C2. Flujo de trabajo del Estudiante: El estudiante es el usuario principal del sistema y el beneficiario directo de su funcionalidad. Su interacción comienza en el entorno de Cadence Virtuoso, donde durante el proceso de diseño de celdas estándar ejecuta la verificación DRC y obtiene uno o más errores en el log generado por Calibre. Con ese error en mano, el estudiante accede a la interfaz del asistente e ingresa la consulta. El sistema procesa la solicitud internamente mediante el pipeline RAG y devuelve una explicación en lenguaje natural del error junto con los pasos específicos para corregirlo en

Virtuoso. Si la corrección resuelve el problema, el estudiante continúa con su diseño. En caso contrario, tiene la posibilidad de reformular la consulta para obtener una orientación más precisa. Si el error está fuera del alcance definido del sistema, el estudiante es dirigido a consultar directamente al profesor o al asistente del curso.

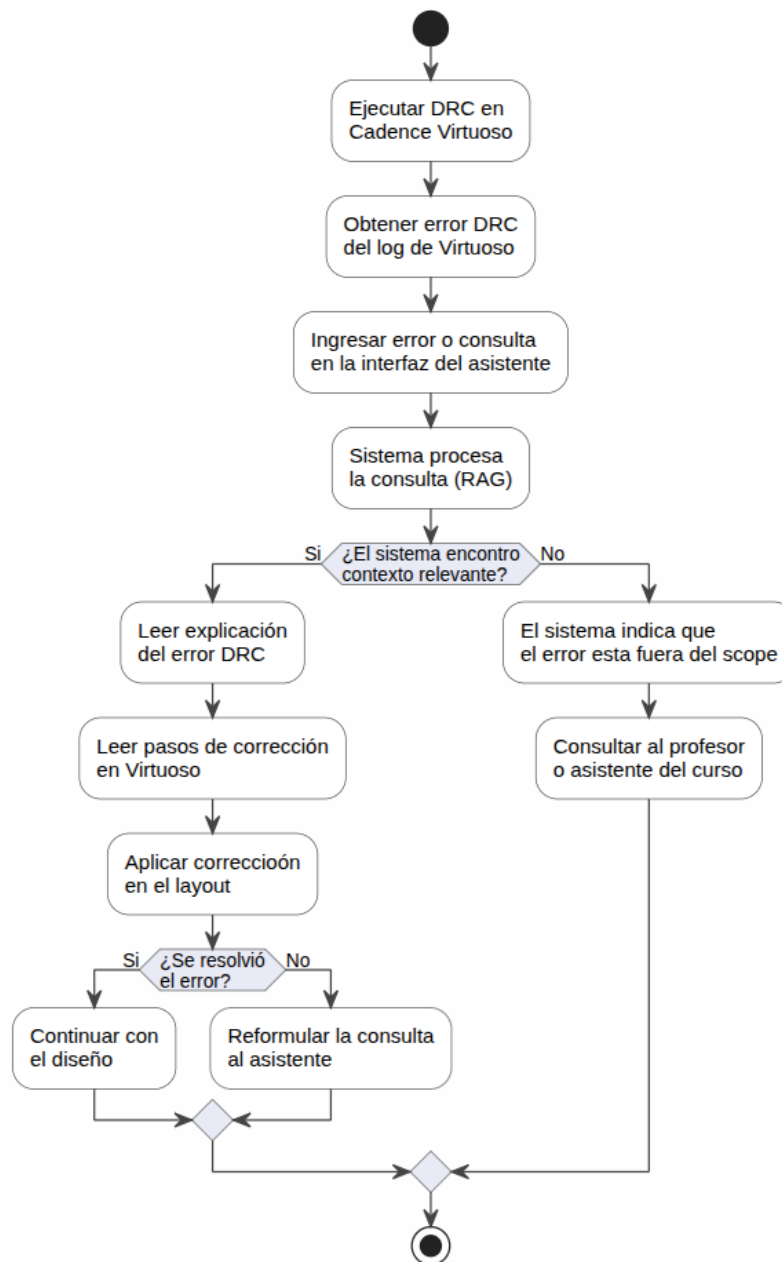


Figura 4. Flujo de trabajo del estudiante

V-C3. Flujo de trabajo del Profesor: El profesor asume un rol de validación y supervisión del sistema. Su participación no es de uso cotidiano sino de evaluación periódica de la calidad de las respuestas generadas. Para ello, selecciona errores DRC cuya solución correcta conoce con certeza y los ingresa al sistema como consultas de prueba. Al recibir la respuesta, la compara con la solución esperada para determinar si el sistema responde de forma correcta y completa. Si la respuesta es satisfactoria, el sistema queda validado para ese caso de uso. De lo contrario, el profesor documenta la deficiencia y notifica al asistente del curso con el detalle necesario para que el corpus sea corregido, iniciando así el ciclo de mejora continua del sistema. Este rol es especialmente relevante durante las primeras semanas de operación, cuando la base de conocimiento aún está en proceso de consolidación.

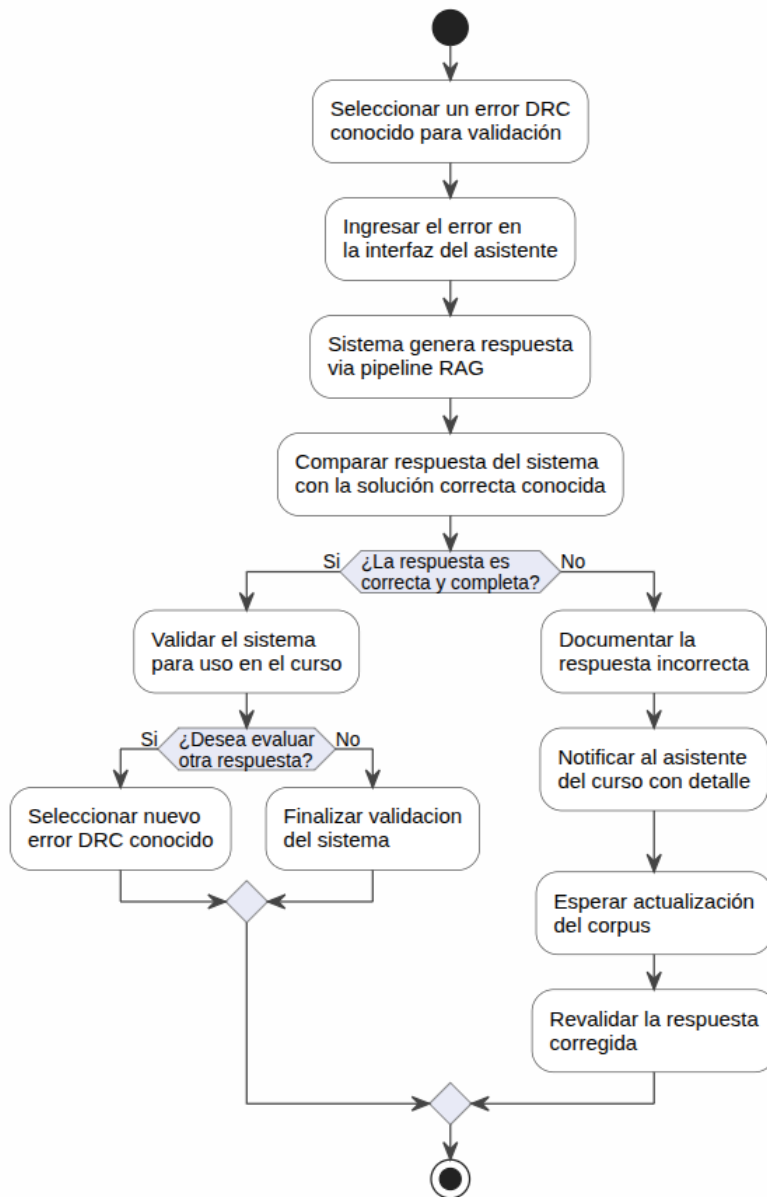


Figura 5. Flujo de trabajo del profesor

VI. VISTA FUNCIONAL DEL SISTEMA

La vista funcional describe la descomposición del sistema en bloques funcionales de software, derivados del análisis de requerimientos y del concepto de operaciones propuesto. Cada bloque encapsula un conjunto cohesivo de responsabilidades y se relaciona con los demás mediante flujos de datos claramente definidos. El sistema se organiza en cuatro grupos funcionales principales: gestión del conocimiento DRC, pipeline RAG, modelo de lenguaje e interfaz de usuario.

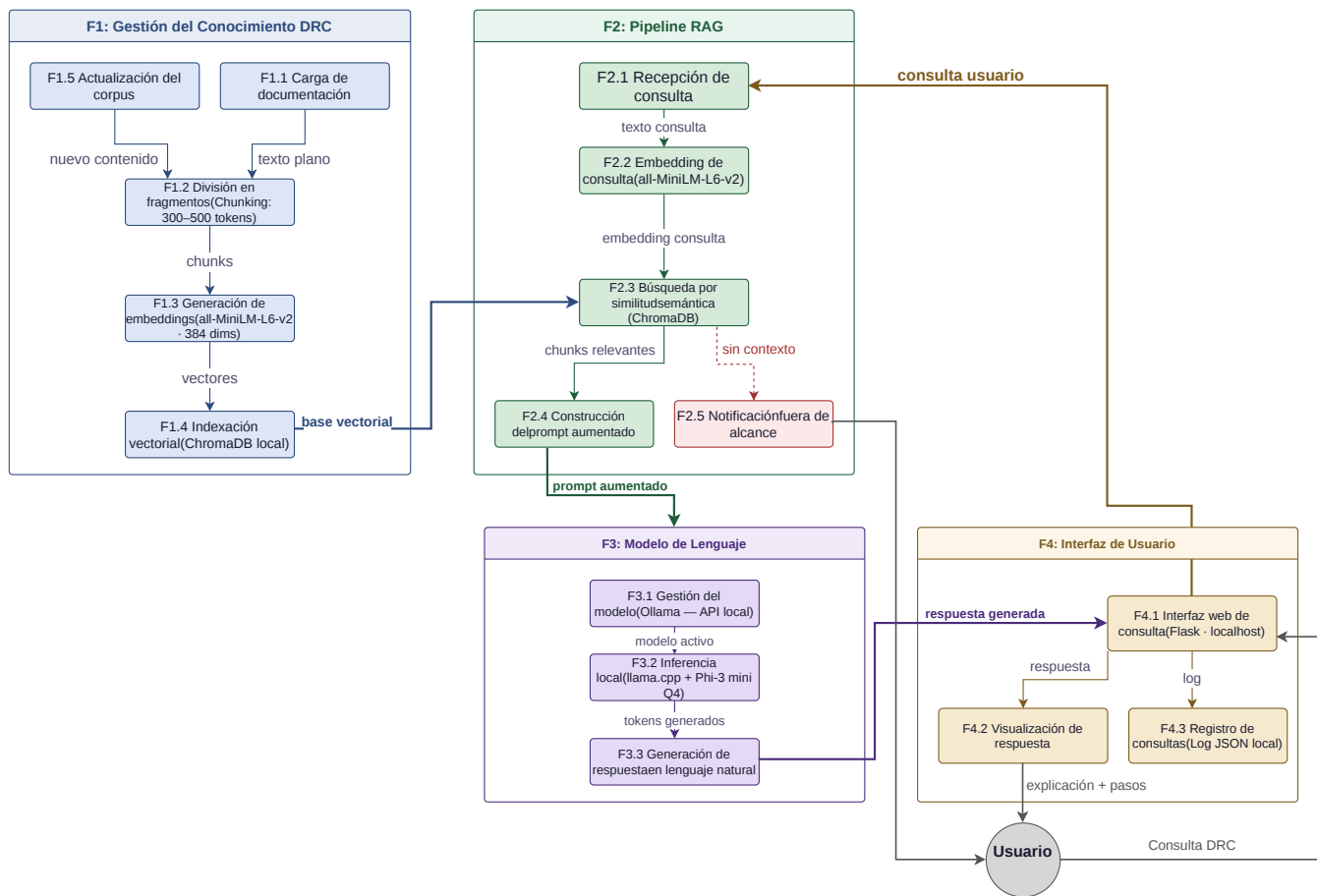


Figura 6. Vista funcional del sistema

VI-A. F1 — Gestión del conocimiento DRC

Este grupo funcional es responsable de construir y mantener la base de conocimiento sobre la cual opera el sistema. La función **F1.1 Carga de documentación** recibe archivos en formato PDF o texto plano proporcionados por el asistente del curso, incluyendo las reglas del PDK XFAB XH018 y ejemplos de errores DRC anotados. La función **F1.2 División en fragmentos** procesa el texto cargado y lo segmenta en unidades de entre 300 y 500 tokens con solapamiento controlado, garantizando que cada fragmento contenga información coherente sobre una regla o ejemplo específico. La función **F1.3 Generación de embeddings** convierte cada fragmento en una representación vectorial de 384 dimensiones mediante el modelo `all-MiniLM-L6-v2`, codificando su significado semántico. La función **F1.4 Indexación vectorial** almacena de forma persistente los fragmentos y sus vectores en ChromaDB, habilitando la recuperación eficiente por similitud. Finalmente, la función **F1.5 Actualización del corpus** permite incorporar nuevo contenido de forma incremental sin reconstruir la base de datos completa, activando nuevamente el flujo de división e indexación.

VI-B. F2 — Pipeline RAG

Este grupo funcional constituye el núcleo del sistema y es responsable de procesar cada consulta del usuario y recuperar el contexto relevante para su respuesta. La función **F2.1 Recepción de consulta** recibe la entrada del usuario desde la interfaz de terminal, ya sea en forma de texto libre o de un error DRC copiado directamente del log de Calibre. La función **F2.2 Embedding de consulta** convierte la consulta en un vector numérico utilizando el mismo modelo de embeddings empleado durante la indexación, asegurando compatibilidad en el espacio vectorial. La función **F2.3 Búsqueda por similitud semántica** compara el vector de la consulta con los vectores almacenados en ChromaDB y recupera los fragmentos más relevantes. Si no se encuentra contexto relevante, la función **F2.5 Notificación fuera de alcance** informa explícitamente al usuario que la consulta excede el dominio definido del sistema. En caso contrario, la función **F2.4 Construcción del prompt aumentado** combina los fragmentos recuperados con la consulta original para construir el prompt que será enviado al modelo de lenguaje.

VI-C. F3 — Modelo de lenguaje

Este grupo funcional es responsable de la generación de la respuesta en lenguaje natural a partir del prompt aumentado. La función **F3.1 Gestión del modelo** utiliza Ollama para administrar el ciclo de vida del modelo Phi-3 mini, incluyendo su carga en memoria y su exposición mediante una API local. La función **F3.2 Inferencia local** ejecuta el proceso de generación de tokens mediante `llama.cpp` directamente sobre la Jetson Nano, operando con el modelo cuantizado en formato Q4 para ajustarse a las restricciones de memoria del hardware. La función **F3.3 Generación de respuesta** produce el texto final en lenguaje natural, que incluye la explicación del error DRC consultado y los pasos específicos para su corrección en Cadence Virtuoso, basándose exclusivamente en el contexto proporcionado por el pipeline RAG.

VI-D. F4 — Interfaz de usuario

Este grupo funcional gestiona la interacción entre el sistema y sus usuarios. La función **F4.1 Interfaz de terminal** expone una interfaz de línea de comandos accesible directamente desde la terminal de la Jetson Nano, desde la cual el estudiante, el profesor o el asistente del curso pueden ingresar consultas y visualizar las respuestas generadas sin requerir software adicional en las estaciones de trabajo. La función **F4.2 Visualización de respuesta** presenta la respuesta generada por el modelo de lenguaje de forma clara y estructurada en la terminal, diferenciando la explicación del error de los pasos de corrección. La función **F4.3 Registro de consultas** almacena de forma local cada consulta recibida junto con la respuesta generada, construyendo un historial que permite al asistente del curso y al profesor identificar patrones de errores frecuentes y evaluar la calidad del sistema a lo largo del semestre.

VII. ARQUITECTURA DEL SISTEMA

La arquitectura del sistema describe cómo las funcionalidades identificadas en la vista funcional se mapean sobre los componentes concretos de hardware y software que conforman la solución. Todo el sistema reside y opera íntegramente sobre la plataforma Jetson Nano de 4 GB de memoria RAM, sobre la cual corre una imagen de Linux sintetizada mediante Yocto Project. El usuario interactúa con el sistema directamente a través de la terminal del dispositivo, sin requerir hardware adicional ni conectividad de red. La arquitectura se organiza en seis capas con responsabilidades claramente delimitadas.

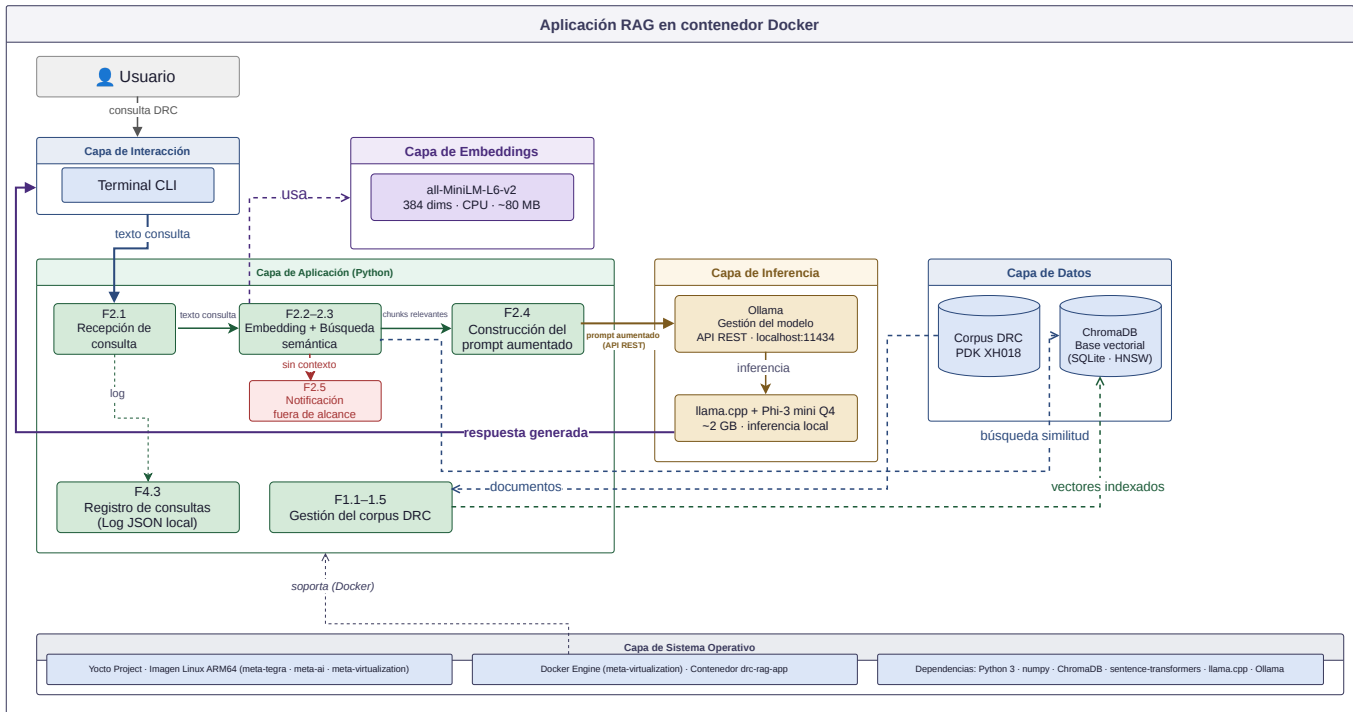


Figura 7. Arquitectura del sistema

VII-A. Capa de interacción

Esta capa representa el punto de contacto entre el usuario y el sistema. El estudiante, el profesor o el asistente del curso interactúan directamente con la terminal CLI de la Jetson Nano, ingresando consultas en lenguaje natural o errores DRC

copiados del log de Calibre. La respuesta generada por el sistema se presenta igualmente en la terminal, de forma estructurada y legible. Esta decisión de diseño elimina dependencias de frameworks de interfaz gráfica y reduce el consumo de recursos del sistema, priorizando la disponibilidad de memoria para el modelo de lenguaje y el pipeline RAG.

VII-B. Capa de aplicación

Esta capa contiene la lógica principal del sistema, implementada en Python. El módulo de recepción de consulta (F2.1) captura la entrada del usuario desde la terminal y la envía al pipeline RAG. El módulo de pipeline RAG (F2.2–F2.3) coordina la generación del embedding de la consulta y la búsqueda por similitud semántica en ChromaDB. El módulo de construcción del prompt aumentado (F2.4) combina los fragmentos recuperados con la consulta original para formar el prompt que se enviará al modelo de lenguaje. El módulo de registro de consultas (F4.3) almacena cada interacción en un log local para revisión posterior. Adicionalmente, el módulo de gestión del corpus (F1.1–F1.5) opera en esta capa cuando el asistente del curso realiza tareas de mantenimiento de la base de conocimiento.

VII-C. Capa de datos

Esta capa aloja los dos repositorios de información del sistema. El corpus DRC contiene los documentos fuente en formato texto, incluyendo las reglas del PDK XFAB XH018 y los ejemplos de errores anotados por el asistente del curso. ChromaDB opera como base de datos vectorial persistente, almacenando los embeddings de los fragmentos generados durante la indexación y proporcionando la capacidad de búsqueda por similitud semántica requerida por el pipeline RAG. Ambos repositorios residen en el almacenamiento local de la Jetson Nano.

VII-D. Capa de embeddings

Esta capa encapsula el modelo `all-MiniLM-L6-v2`, responsable de convertir texto en vectores numéricos de 384 dimensiones. El modelo opera exclusivamente en CPU, con un tamaño aproximado de 80 MB, lo que lo hace compatible con las restricciones de memoria de la Jetson Nano. Este mismo modelo se utiliza tanto durante la indexación del corpus como durante el procesamiento de cada consulta, garantizando consistencia en el espacio vectorial de búsqueda.

VII-E. Capa de inferencia

Esta capa contiene los componentes responsables de la generación de texto. Ollama actúa como gestor del ciclo de vida del modelo de lenguaje, administrando su carga en memoria y exponiendo una API local que el pipeline RAG utiliza para enviar el prompt aumentado. El modelo Phi-3 mini, ejecutado mediante `llama.cpp` en su versión cuantizada Q4, realiza la inferencia local generando la respuesta en lenguaje natural a partir del prompt recibido. La cuantización Q4 reduce el tamaño del modelo a aproximadamente 2 GB, permitiendo su coexistencia en memoria con los demás componentes del sistema dentro del límite de 4 GB de la Jetson Nano.

VII-F. Capa de sistema operativo

Esta capa constituye la base sobre la cual operan todos los componentes anteriores. La imagen de Linux es sintetizada mediante Yocto Project, lo que permite construir un sistema operativo mínimo y optimizado para el hardware de la Jetson Nano, incluyendo únicamente los paquetes necesarios para la operación del sistema. Las dependencias de software incluyen Python, las bibliotecas del pipeline RAG, ChromaDB, el modelo de embeddings y `llama.cpp`, todas integradas como recetas dentro del flujo de trabajo de Yocto Project conforme a los requerimientos FR-12 y FR-13.

VIII. ANÁLISIS DE DEPENDENCIAS

El análisis de dependencias presenta los componentes de software necesarios para construir la imagen del sistema operativo mediante Yocto Project, así como sus relaciones estructurales. La arquitectura se estructura en dos fases claramente diferenciadas: una fase de compilación mediante Yocto Project, donde se ensamblan las capas de soporte base y herramientas del sistema, y una fase de post-despliegue en Jetson Nano, donde se descargan los modelos de inteligencia artificial. Esta separación permite una imagen optimizada, reproducible y mantenible, además de reducir significativamente el tamaño de la imagen compilada.

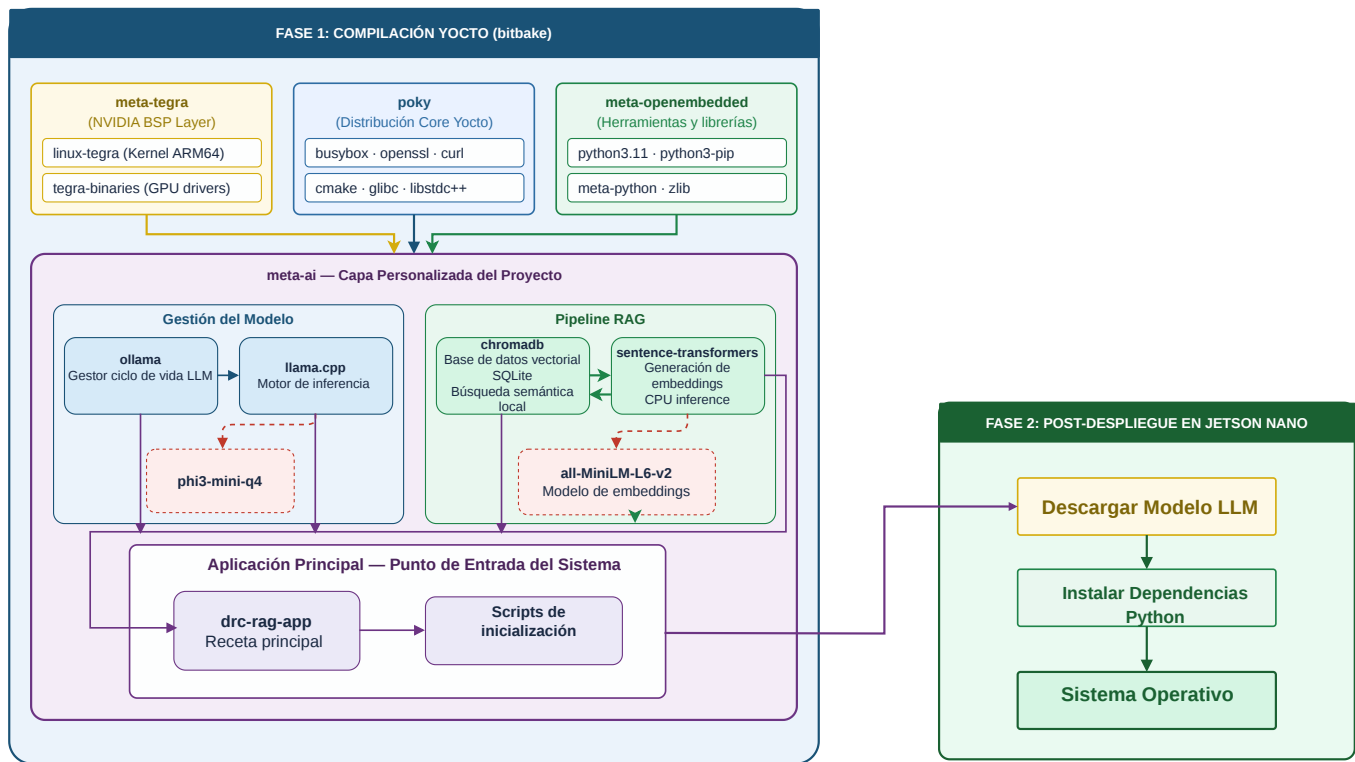


Figura 8. Árbol de dependencias de software para la imagen Yocto

VIII-A. meta-tegra (BSP Layer)

Esta capa provee el soporte de placa base (Board Support Package) para la Jetson Nano. Incluye el kernel `linux-tegra`, optimizado para la arquitectura ARM de la plataforma, y los binarios propietarios de NVIDIA (`tegra-binaries`) necesarios para la gestión de la GPU y los drivers de hardware específicos del dispositivo. Esta capa es el punto de partida obligatorio para cualquier imagen Yocto destinada a la Jetson Nano.

VIII-B. poky (core)

La capa `poky` es la distribución base de Yocto Project y provee los componentes fundamentales del sistema operativo. Para este proyecto se requieren `busybox` como utilidad base del sistema, `openssl` para operaciones criptográficas requeridas por Ollama, `curl` para la descarga de modelos y `cmake` como sistema de construcción necesario para compilar `llama.cpp`.

VIII-C. meta-openembedded

Esta capa extiende `poky` con paquetes adicionales de software abierto. Para el sistema se requieren las siguientes recetas: `python3` como intérprete base del pipeline RAG, `python3-pip` para la gestión de paquetes Python dentro de la imagen, `python3-numpy` como dependencia fundamental de los componentes de machine learning, y las bibliotecas del sistema `glibc` y `libstdc++` necesarias para la ejecución de componentes compilados en C++.

VIII-D. meta-ai (capa personalizada)

Esta es la capa central del proyecto, desarrollada por el equipo para integrar todos los componentes de inteligencia artificial del sistema. Se organiza en cuatro grupos de recetas:

Gestión del modelo: La receta `ollama` gestiona el ciclo de vida del modelo de lenguaje y depende de `llama.cpp` para la ejecución de la inferencia. La receta `llama.cpp` compila el motor de inferencia en C++ y requiere `cmake` y `libstdc++`. El modelo `phi3-mini-q4` se distribuye en formato GGUF y es cargado por `llama.cpp` en tiempo de ejecución.

Pipeline RAG: La receta `chromadb` provee la base de datos vectorial local y depende de `python3` y `numpy`. La receta `sentence-transformers` provee la lógica de generación de embeddings y opera exclusivamente en CPU, sin dependencia de frameworks de aprendizaje profundo de gran peso como PyTorch. Sus dependencias de compilación son `python3` y `numpy`. El modelo `all-MiniLM-L6-v2`, utilizado para la generación de embeddings de 384 dimensiones, no es empaquetado en la imagen Yocto; su descarga desde Hugging Face Hub se realiza durante la fase de post-despliegue sobre la Jetson Nano, separando así los artefactos de inteligencia artificial del proceso de compilación reproducible.

Motor de contenedores: La receta `docker-engine`, proveniente de la capa `meta-virtualization` de OpenEmbedded, integra el motor de contenedores Docker en la imagen Yocto. Esta receta habilita la ejecución de la aplicación RAG como contenedor sobre el sistema operativo sintetizado, separando el ciclo de vida del sistema operativo del ciclo de vida de la aplicación. Sus dependencias de sistema son `python3` y `libstdc++`.

Aplicación RAG: La receta `rag-pipeline` es la receta principal del proyecto. Integra todos los componentes anteriores y contiene el código del pipeline RAG, el módulo de gestión del corpus DRC y la interfaz de terminal. Esta receta es el punto de entrada del sistema y depende directamente de `ollama`, `chromadb`, `sentence-transformers` y `python3`.

IX. ESTRATEGIA DE INTEGRACIÓN

La estrategia de integración describe el proceso completo mediante el cual los componentes de hardware y software del sistema se ensamblan en una solución funcional y desplegable. El proceso se organiza en tres fases secuenciales y claramente diferenciadas: compilación del sistema operativo, post-despliegue de componentes de inteligencia artificial, y operación en el entorno de laboratorio. La figura 9 ilustra el flujo completo de integración.

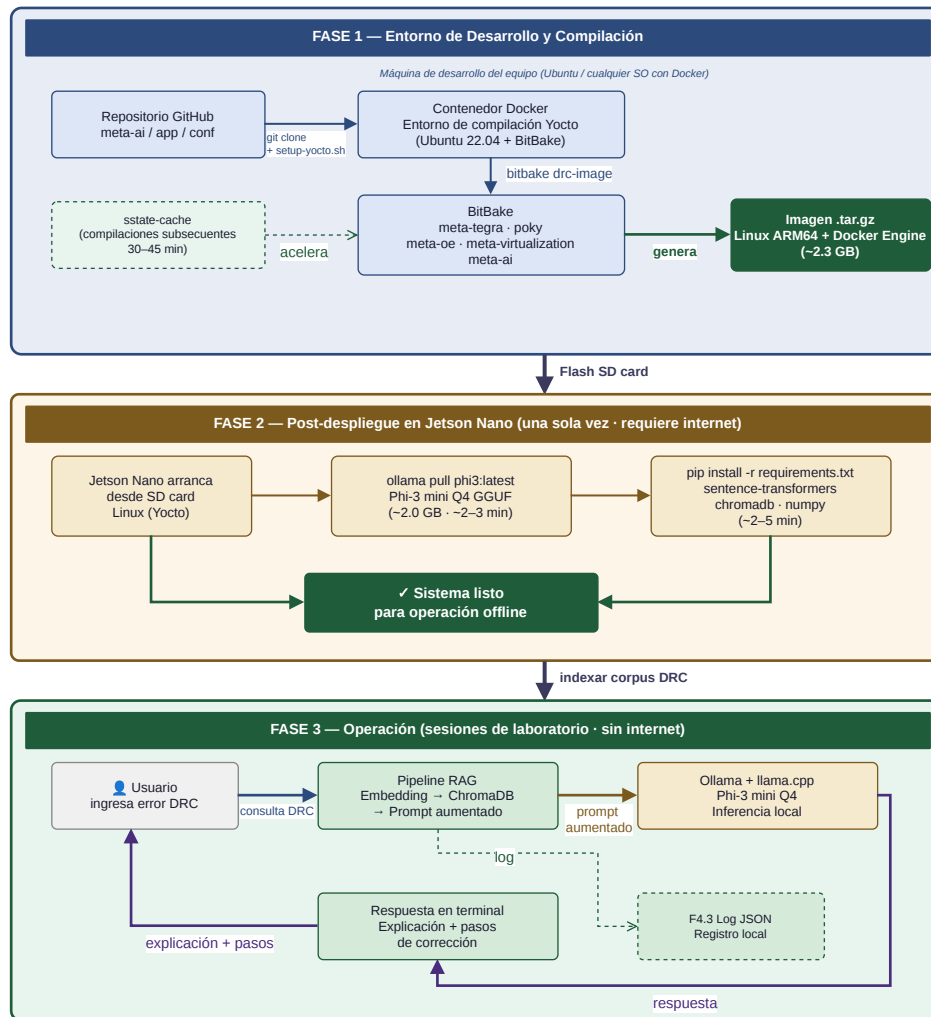


Figura 9. Estrategia de integración del sistema en tres fases

IX-A. Fase 1 — Compilación con Yocto Project

La primera fase tiene lugar en la máquina de desarrollo del equipo y produce la imagen de Linux que será instalada en la Jetson Nano. Para garantizar determinismo y reproducibilidad entre los entornos de desarrollo de los integrantes del equipo, el proceso de compilación se ejecuta íntegramente dentro de un contenedor Docker configurado con Ubuntu 22.04 y las herramientas de Yocto Project. Este enfoque elimina la dependencia del sistema operativo host y asegura que todos los miembros del equipo obtengan resultados idénticos independientemente de su configuración local.

El flujo de compilación inicia con la clonación del repositorio del proyecto desde GitHub, el cual contiene la capa personalizada `meta-ai` con las recetas de los componentes de inteligencia artificial, los archivos de configuración del sistema y el script de inicialización `setup-yocto.sh`. Este script automatiza la descarga de las capas base de Yocto: `poky`, `meta-tegra`, `meta-openembedded` y `meta-virtualization`. Una vez disponibles todas las capas, se invoca `bitbake drc-image`, que orquesta la compilación cruzada de todos los paquetes para la arquitectura ARM64 de la Jetson Nano.

El resultado de esta fase es una imagen `.tar.gz` de aproximadamente 2.3 GB que contiene el kernel Linux optimizado para la plataforma, el motor de contenedores Docker Engine, las herramientas del pipeline RAG compiladas para ARM64 y todos los scripts de inicialización del sistema. La duración de la compilación es de 2 a 4 horas en la primera ejecución; las compilaciones subsecuentes se reducen a 30–45 minutos gracias al mecanismo de caché `sstate-cache` de Yocto Project.

IX-B. Fase 2 — Post-despliegue en Jetson Nano

La segunda fase se ejecuta una única vez sobre la Jetson Nano, inmediatamente después de instalar la imagen generada en la tarjeta SD y realizar el primer arranque del sistema. Esta fase requiere conectividad a internet y tiene una duración aproximada de 5 a 10 minutos.

El proceso consiste en dos pasos secuenciales. El primero es la descarga del modelo de lenguaje mediante el comando `ollama pull phi3:latest`, que obtiene el modelo Phi-3 mini en formato GGUF con cuantización Q4 desde los servidores de Ollama. Este modelo ocupa aproximadamente 2.0 GB en disco y no se incluye en la imagen Yocto para mantener el tamaño de la misma dentro de límites manejables. El segundo paso es la instalación de las dependencias Python del pipeline RAG mediante `pip install -r requirements.txt`, que incluye `sentence-transformers`, `chromadb` y `numpy`. Adicionalmente, en este paso se descarga automáticamente el modelo de embeddings `all-MiniLM-L6-v2` desde Hugging Face Hub, con un tamaño aproximado de 80 MB.

Al finalizar esta fase, el sistema dispone de todos los componentes necesarios para operar de forma completamente autónoma y sin conexión a internet.

IX-C. Fase 3 — Operación en el laboratorio

La tercera fase corresponde al ciclo de operación normal del sistema durante las sesiones de laboratorio. Esta fase no requiere intervención técnica por parte del equipo de desarrollo y opera de forma completamente offline.

Antes del inicio del semestre, el asistente del curso ejecuta el proceso de indexación del corpus DRC, cargando la documentación técnica del PDK XFAB XH018 al sistema mediante la interfaz de terminal. El pipeline RAG procesa los documentos, genera los embeddings mediante `all-MiniLM-L6-v2` y almacena los vectores resultantes en ChromaDB. Este proceso se realiza una única vez por semestre, con actualizaciones incrementales según las necesidades del curso.

Durante las sesiones, el ciclo de operación por consulta es el siguiente: el estudiante ingresa un error DRC o una pregunta en lenguaje natural a través de la terminal de la Jetson Nano; el pipeline RAG genera el embedding de la consulta, recupera los fragmentos más relevantes de ChromaDB y construye el prompt aumentado; Ollama envía el prompt al motor de inferencia `llama.cpp`, que ejecuta el modelo Phi-3 mini Q4 localmente; la respuesta generada se presenta al estudiante en la terminal con la explicación del error y los pasos de corrección, y la consulta queda registrada en el log local del sistema. Todo este ciclo se completa en un tiempo máximo de 60 segundos, conforme al requerimiento NFR-01.

IX-D. Repositorio y colaboración del equipo

El repositorio público en GitHub centraliza todos los artefactos versionables del proyecto. La estructura del repositorio separa el código de la aplicación Python (`app/`), las recetas Yocto de la capa personalizada (`yocto/meta-ai/`) y la documentación del proyecto (`docs/`). Los artefactos de gran tamaño que no deben versionarse —incluyendo los directorios de compilación de Yocto, los modelos descargados y las imágenes generadas— se excluyen mediante el archivo `.gitignore`. El script `setup-yocto.sh` garantiza que cualquier integrante del equipo pueda reproducir el entorno de compilación completo a partir exclusivamente del contenido del repositorio.

X. CRONOGRAMA

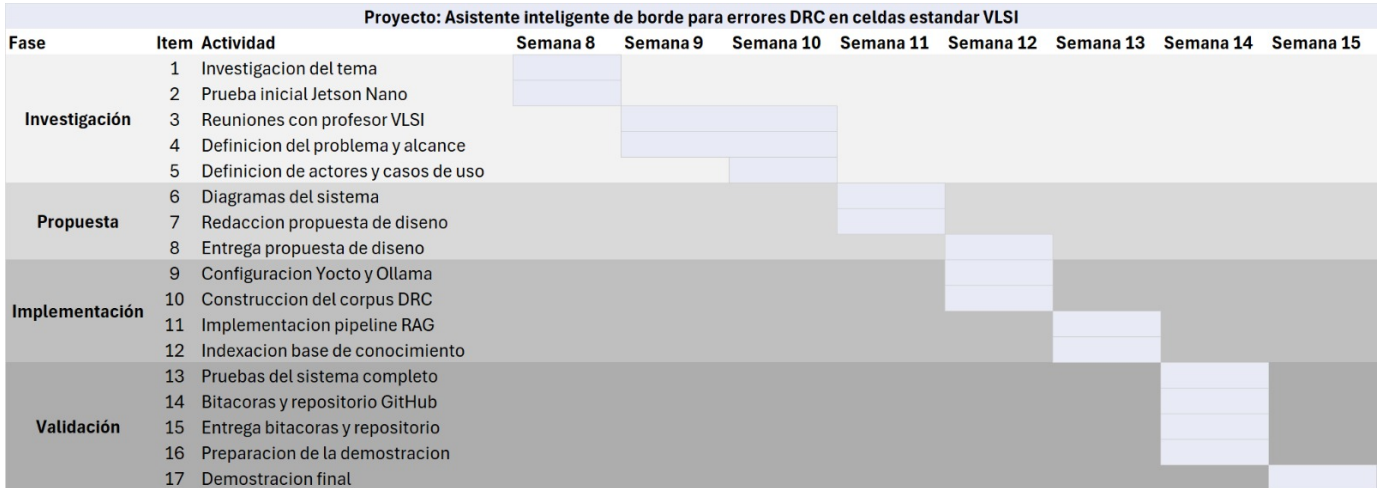


Figura 10. Cronograma de trabajo del proyecto para el desarrollo del asistente inteligente de borde

XI. CONCLUSIONES

La propuesta de diseño presentada para el desarrollo de un asistente inteligente de borde se fundamenta en una arquitectura técnica coherente que equilibra las capacidades de razonamiento de los modelos de lenguaje modernos con las restricciones físicas de la computación local. A partir del análisis del diseño, se concluye que la implementación teórica de la arquitectura Generación Aumentada por Recuperación (RAG) es viable para la plataforma NVIDIA Jetson Nano, ya que el uso de modelos pequeños como Phi-3 mini con cuantización de 4 bits permite que el sistema opere dentro del límite de 4 GB de RAM, dejando margen suficiente para la base de datos vectorial ChromaDB y los servicios de inferencia. Desde una perspectiva operativa, el diseño justifica plenamente la transición hacia la Edge AI como un mecanismo para garantizar la privacidad y soberanía de los datos técnicos del PDK XFAB XH018, eliminando la dependencia de la nube y asegurando una operación estable en el entorno del laboratorio. La robustez metodológica de la propuesta destaca por la integración de Yocto Project para la síntesis de un sistema operativo personalizado y optimizado, lo que asegura que el software final sea reproducible y eficiente, evitando el sobre costo de recursos de las imágenes de Linux genéricas. Finalmente, la propuesta proyecta un impacto pedagógico significativo al identificar y abordar una barrera crítica en el aprendizaje del diseño VLSI, proponiendo una herramienta que permitiría a los estudiantes resolver dudas técnicas de forma autónoma y en tiempo real. En conjunto, el diseño detallado, los requerimientos establecidos y la estrategia de integración en fases demuestran que el proyecto tiene una base sólida y madura para proceder a su ejecución, cumpliendo con los estándares de rendimiento, sostenibilidad y seguridad exigidos para un sistema embebido de nueva generación.

REFERENCIAS

- [1] J. Rivera Mancilla, S. Zapata Cortés, R. Pizarro Carreño, and B. Keith Norambuena, "Enhancing chatbot performance with retrieval augmented generation and prompt engineering," in *Workshop on Data and Knowledge Engineering (WDKE'24)*, Copiapó, Chile, 2024.
- [2] rlabconnect, "Nvidia-jetson-nano-rag-chatbot," <https://github.com/rlabconnect/NVIDIA-Jetson-Nano-RAG-Chatbot>, 2026.
- [3] Intel Corporation, "What is edge ai?" 2025. [Online]. Available: <https://www.intel.com/content/www/us/en/learn/edge-ai.html>
- [4] J. Diaz-Gorriñ, C. Caballero-Gil, and L. Brankovic, "Enhancing network security with generative ai on jetson orin nano," *Applied Sciences*, vol. 16, no. 01442, 2026.
- [5] M. Abdin *et al.*, "Phi-3 technical report: A highly capable language model locally on your phone," *arXiv preprint arXiv:2404.14219*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.14219>
- [6] M. Abdulkadhim and S. R. Repas, "Introducing leaf: Llm edge assessment framework for generative ai on the edge," *Machine Learning and Knowledge Extraction*, 2026.
- [7] K. Seemakhupt *et al.*, "Edgerag: Online-indexed rag for edge devices," *arXiv preprint arXiv:2412.21023*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.21023>
- [8] Cadence Design Systems, Inc., "Pegasus verification system datasheet: Cloud-ready, massively parallel physical signoff," Cadence Design Systems, Tech. Rep., 2024. [Online]. Available: https://www.cadence.com/en_US/home/resources/datasheets/pegasus-verification-system-ds.html
- [9] J. Kumar, "Ai in vlsi physical design: Opportunities and challenges," 2025. [Online]. Available: <https://www.design-reuse.com/article/61623-ai-in-vlsi-physical-design-opportunities-and-challenges/>
- [10] A. Kaintura, P. R. S. S. Luar, and I. I. Almeida, "Orassistant: A custom rag-based conversational assistant for openroad," *arXiv preprint arXiv:2410.03845*, 2024. [Online]. Available: <https://arxiv.org/abs/2410.03845>